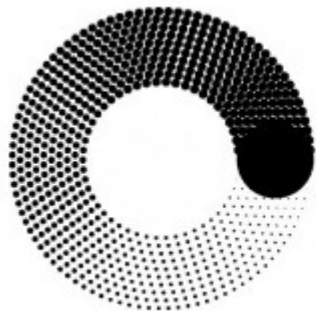




**МОСКОВСКИЙ
ПОЛИТЕХ**

**АВТОМАТИЗАЦИЯ ПРОЦЕССОВ ЖИЗНЕННОГО
ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ**

**Лабораторная работа № 5.
Непрерывная доставка**

**Москва
2024**

Теоретические основы

Непрерывная доставка (Continuous delivery, CD) является расширением непрерывной интеграции, поскольку она автоматически развертывает все изменения кода в тестовой и/или производственной среде после этапа сборки.

Это означает, что помимо автоматического тестирования у вас есть автоматизированный процесс выпуска, и вы можете развернуть свое приложение в любое время, нажав кнопку.

Теоретически при непрерывной доставке возможно выпускать обновления продукта ежедневно, еженедельно, раз в две недели или в зависимости от бизнес-требований организации. Однако, если необходимо действительно воспользоваться преимуществами непрерывной доставки, развертывание в рабочей среде следует выполнять как можно раньше, чтобы убедиться, что изменения между обновлениями минимальны и, в случае проблем, их можно легко откатить.

CD — это комплексная методология для всей организации, которая включает неинженерные команды, такие как дизайнеры, продуктологи и маркетологи.

Автоматизация является ключевой ценностью CD. Человеческое время стоит дорого, и его следует тратить на творческие задачи, а не на утомительное выполнение задач по инструкции. Ручной процесс не является по-настоящему воспроизводимым и надежным, пока он не закреплен за кодом и не может выполняться автоматически по запросу. Автоматизированные задачи могут быть объединены вместе для создания дополнительных уровней автоматизации.

Максимально необходимо автоматизировать: тесты, релизы, изменения конфигурации и многое другое

Задания

1. Добавить к созданным в ходе ЛР4 шагам шаги, осуществляющие развертывание приложения на stage-сервере из образа, загруженного в Docker Hub вместе с базой данных (либо иным сторонним сервисом, добавленным в docker-compose.yaml в ходе ЛР3). Состав шагов и их наполнение выбираются студентом в зависимости от выбранного типа приложения.

2. Развернуть на test-стенде образ jetbrains/teamcity-agent, настроить его для работы с уже запущенным сервером

3. Внести в репозиторий какое-либо изменение, касающееся функционала разрабатываемого приложения, в соответствии с требованиями внесения изменений GitFlow, либо другой методологии на выбор студента. В дальнейшем

все изменения в код репозитория необходимо вносить согласно выбранной методологии.

4. Зафиксировать успешную сборку приложения из кода, добавленного в feature-ветку в TeamCity

5. Создать пул-реквест созданной ветки в ветку dev с требованием согласования со стороны другого члена команды

6. После согласования и создания коммита, зафиксировать сборку приложения из кода в ветке dev и развертывание приложения из созданного образа на stage-сервере

7. Подготовить отчет по лабораторной работе (текстовый документ и скриншоты с описанием по вышеописанным пунктам).